DOCUMENTACIÓN TÉCNICA
Automatización de Compulsa de Repuestos para Seguro Automotor

Entrega Final — Curso de Automatización con IA (Coder)

Stack: n8n · Airtable · Claude · Slack · Gmail

Flujo A — Solicitud de Compulsa a Proveedores

Flujo B — Procesamiento de Respuestas y Cierre de Cotización

Gonzalo Lodeiro · lodeiro.gonzaloc@gmail.com

1. Resumen ejecutivo y arquitectura general

Este documento describe la automatización de punta a punta del proceso de **compulsa de precios de repuestos** para la reparación de vehículos en el marco de un seguro automotor. El objetivo del proyecto es reemplazar un proceso manual — pedir presupuesto a varios proveedores por mail, esperar respuestas y armar la cotización final a mano — por un pipeline automatizado que dispara solo, usa IA para redactar y leer mensajes, y deja dos puntos de control humano (HITL) antes de cualquier acción irreversible (enviar un pedido a un proveedor real, o enviar la cotización final al cliente).

El sistema está dividido en **dos workflows de n8n independientes** que comparten una única base de datos en Airtable como estado central:

- **Flujo A — Solicitud a Proveedores:** se dispara cuando una cotización nueva queda lista (Estado = "Pendiente"), arma con IA el texto de la solicitud de compulsa, pide aprobación humana por Slack antes de escribirle a ningún proveedor real, y envía la solicitud por Gmail solo a los proveedores que corresponden según la marca del vehículo.
- **Flujo B — Procesamiento de Respuestas y Cierre:** se dispara cuando un proveedor responde por mail, usa IA para extraer el precio cotizado, pide aprobación humana por Slack antes de dar ese precio por bueno, y si se aprueba arma y envía el mail final al cliente con el total (repuesto + mano de obra), cerrando la cotización en Airtable.

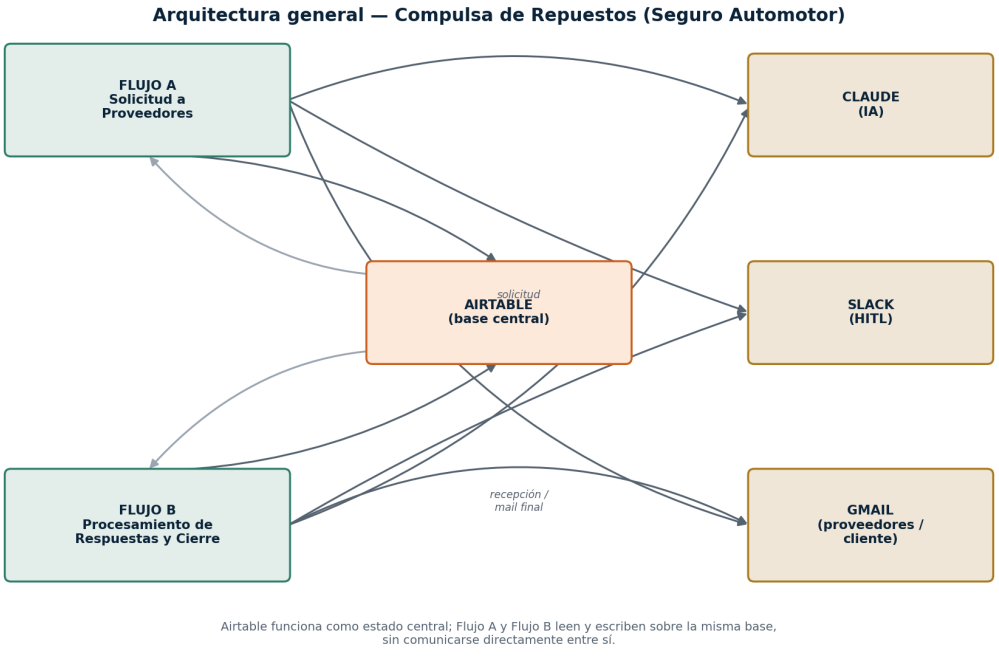


Figura 1. Arquitectura general: Airtable como estado central, dos flujos independientes, tres integraciones externas (Claude, Slack, Gmail).

Por qué dos flujos separados y no uno solo

Flujo A y Flujo B resuelven momentos distintos del proceso y se disparan por eventos distintos (una cotización nueva vs. un mail entrante), así que mantenerlos separados evita un único workflow gigante con múltiples triggers compitiendo, y permite versionar, probar y (des)activar cada mitad del proceso de forma independiente. La comunicación entre ambos no es directa: pasa siempre por el campo **Estado** de la tabla Cotizacion, que funciona como una máquina de estados legible por cualquiera de los dos flujos.

Los dos puntos de control humano (HITL)

Ambos flujos usan el mismo patrón de n8n para pausar y esperar una decisión humana: un nodo Slack en modo **sendAndWait**, que manda un mensaje con botones Aprobar/Rechazar, congela la ejecución del workflow (sin consumir recursos ni depender de un segundo trigger) y la retoma automáticamente apenas alguien responde. Esto es deliberado: en un proceso de seguros, ni el envío de una solicitud a un proveedor real ni el envío de una cotización final al cliente deberían depender 100% de lo que decida un modelo de IA sin supervisión.

2. Modelo de datos en Airtable

Toda la base vive en una única base de Airtable (`appN0kV16XLFXn6RX`), con **Cotizacion** como tabla central y cuatro tablas satélite vinculadas por campos de link: Vehiculos, Clientes, Proveedores y Repuestos.

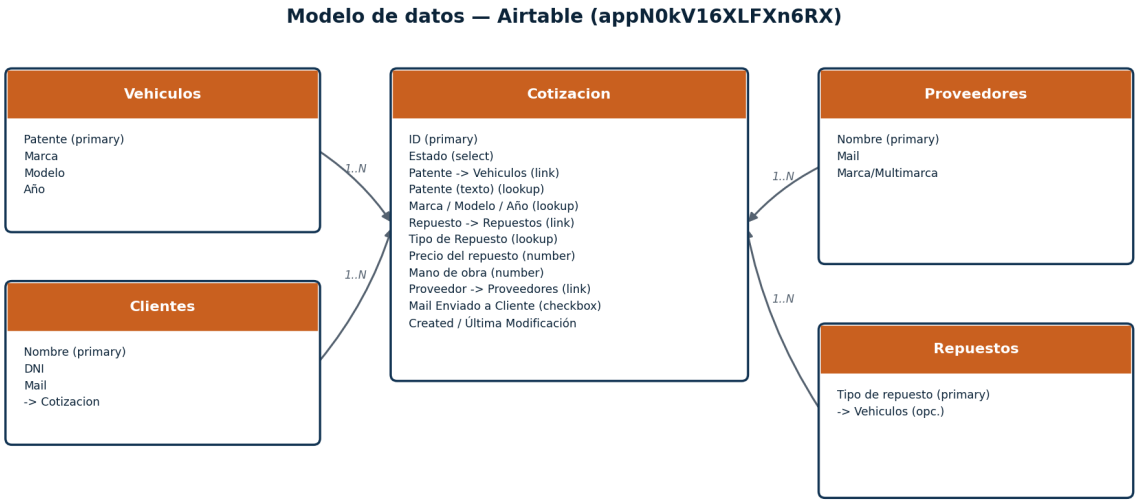


Figura 2. Modelo entidad-relación simplificado de la base.

El patrón "link + lookup" para datos legibles por IA y Slack

Un detalle de diseño que se repite en toda la base, y que fue causa de más de un bug real durante el testing (ver sección 5): cuando un campo de Cotizacion es un *link* a otra tabla (por ejemplo Patente, que linkea a Vehiculos, o Repuesto, que linkea a Repuestos), cualquier expresión que lo lea directamente devuelve el **record ID interno de Airtable** (algo como `rec804IhZyGJlj5p2`), no el valor humano-legible ("Patente1", "Puerta Delantera Derecha"). La solución aplicada en los dos casos donde esto se detectó fue agregar un **campo lookup adicional** en Cotizacion ("Patente (texto)", "Tipo de Repuesto") que trae el valor de texto real a través del link existente, y usar ese lookup — nunca el link crudo — en los prompts de Claude y en los mensajes de Slack.

Campo Estado: la máquina de estados del proceso

El campo **Estado** (singleSelect) de Cotizacion es el que coordina Flujo A y Flujo B sin que se comuniquen directamente: `Pendiente` → `Compulsa preparada` → `Enviado a proveedores` → `Procesado por IA` → `Aprobado por Humano` (con las ramas alternativas "Rechazado / Ajustado por Humano" en caso de rechazo). Cada flujo solo actúa sobre las filas cuyo Estado corresponde a su propia etapa, y escribe el siguiente estado al terminar — es, en la práctica, una cola de trabajo implementada sobre un campo de Airtable.

3. Flujo A — Solicitud de Compulsa a Proveedores

Se dispara con el Airtable Trigger **"Nuevo registro en Cotizacion"**, sondeando el campo `Última Modificación` cada minuto (no `Created` — ver nota de diseño más abajo). Un guard anti-duplicado inmediatamente después del trigger asegura que solo avance la compulsa cuando el registro está realmente completo.

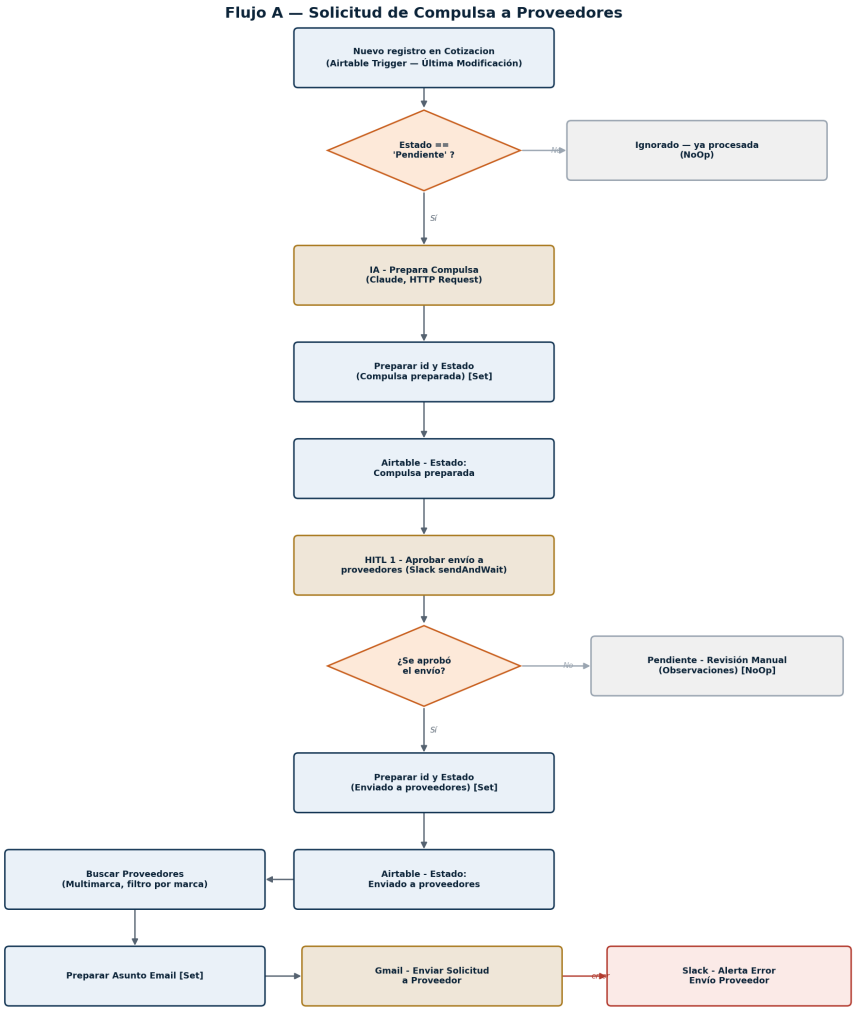


Figura 3. Diagrama de nodos de Flujo A.

Detalle de nodos

Nodo	Tipo	Función
Nuevo registro en Cotizacion	Airtable Trigger	Sondea la tabla Cotizacion cada minuto sobre el campo Última Modificación.
IF - Cotizacion sin procesar (Anti-duplicado)	IF	Solo deja pasar el registro si Estado == "Pendiente" exacto; evita procesar filas a medio cargar o ya avanzadas.
Ignorado - Cotizacion ya procesada	NoOp	Rama falsa del guard anterior; termina el flujo sin hacer nada.
IA - Prepara Compulsa (Claude)	HTTP Request	Redacta el texto de la solicitud de compulsa a partir de Patente, Marca, Modelo y Tipo de Repuesto (vía los lookups, nunca los links crudos).

Nodo	Tipo	Función
Preparar id y Estado (Compulsa preparada)	Set	Arma id + Estado="Compulsa preparada" para el update siguiente.
Airtable - Estado: Compulsa preparada	Airtable Update	Escribe el nuevo estado en el registro.
HITL 1 - Aprobar envío a proveedores	Slack (sendAndWait)	Primer punto de control humano: pausa la ejecución hasta que alguien aprueba o rechaza el envío real a proveedores.
¿Se aprobó el envío?	IF	Evalúa $json.data.approved === true, tomado directamente de la respuesta del botón de Slack.
Pendiente - Revisión Manual (Observaciones)	NoOp	Rama de rechazo; el registro queda para revisión manual.
Preparar id y Estado (Enviado a proveedores)	Set	Arma el siguiente estado tras la aprobación.
Airtable - Estado: Enviado a proveedores	Airtable Update	Escribe el estado y devuelve el registro completo (incluida la Marca, vía lookup) para el filtro siguiente.
Buscar Proveedores (Multimarca)	Airtable Search	Filtra proveedores por fórmula: Multimarca, o marca exacta igual a la del vehículo — evita mandar el mismo pedido a proveedores que no corresponden.
Preparar Asunto Email	Set	Arma el asunto ("COT-") para el mail de solicitud.
Gmail - Enviar Solicitud a Proveedor	Gmail Send	Envía la solicitud real, uno por cada proveedor que superó el filtro.
Slack - Alerta Error Envío Proveedor	Slack	Rama de error del nodo Gmail; avisa si el envío falla.

Decisión de diseño: por qué el trigger mira "Última Modificación" y no "Created"

El trigger originalmente escuchaba el campo `Created` (fecha de creación de la fila). El problema real, encontrado en testing con datos reales: Airtable fija ese valor en el instante exacto en que aparece la fila nueva en la grilla, antes de que se termine de completar ningún otro campo — así que si alguien está cargando una cotización a mano, el trigger puede dispararse sobre un registro todavía vacío, y como `Created` nunca cambia, ese registro no se vuelve a disparar solo aunque después se complete. Se corrigió apuntando el trigger a `Última Modificación` (que sí cambia con cada edición) y endureciendo el guard anti-duplicado para exigir Estado == "Pendiente" exacto — así el flujo de carga correcto queda: completar todos los campos primero, y recién al final poner Estado = "Pendiente" para disparar la compulsa.

4. Flujo B — Procesamiento de Respuestas y Cierre de Cotización

Se dispara con un Gmail Trigger apenas un proveedor responde a la solicitud de Flujo A. Está dividido en dos ramas conectadas en una sola ejecución: **Rama 1** (leer la respuesta, extraer el precio con IA, pedir validación humana) y **Rama 2** (armar y enviar el mail final al cliente, cerrar la cotización). Rama 2 fue **rediseñada** durante el testing final: originalmente dependía de que alguien editara el campo Estado a mano en Airtable para avanzar; ahora usa el mismo patrón sendAndWait de Slack que Flujo A, así que la aprobación del precio por botón dispara el cierre automáticamente, sin pasos manuales.

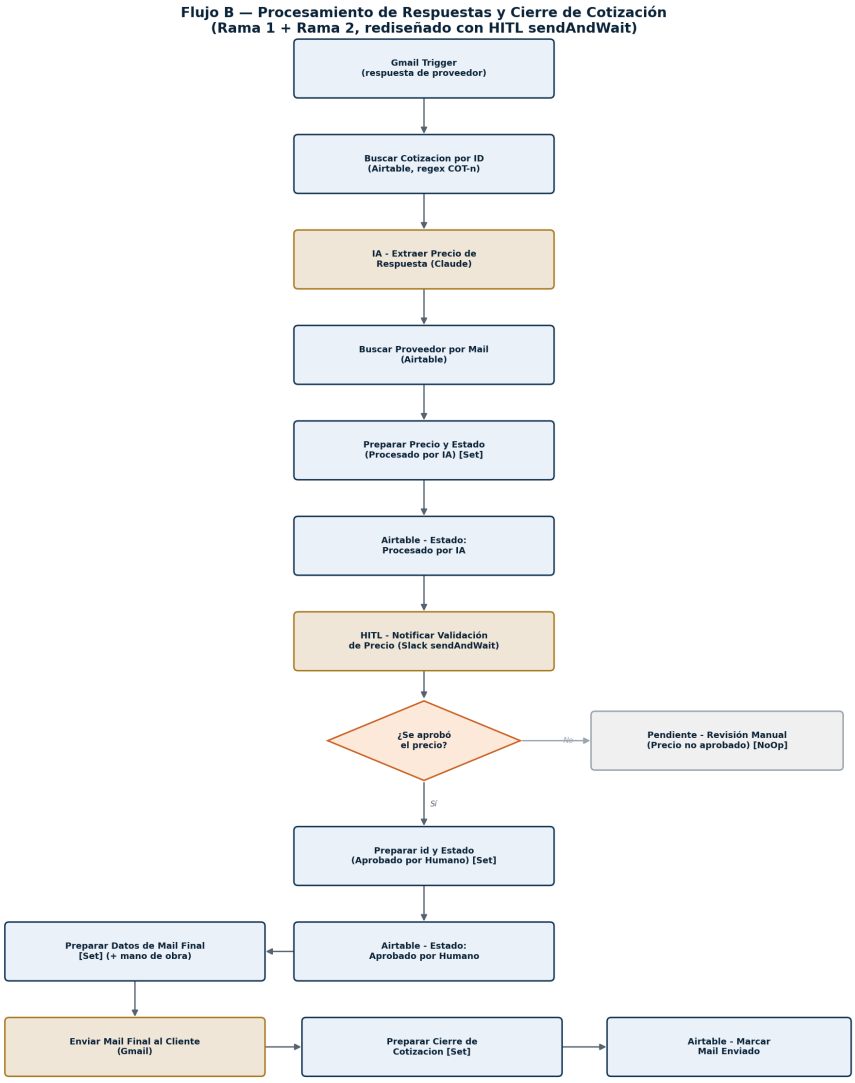


Figura 4. Diagrama de nodos de Flujo B (versión rediseñada).

Detalle de nodos

Nodo	Tipo	Función
Gmail Trigger	Gmail Trigger	Detecta la respuesta del proveedor por mail.
Buscar Cotizacion por ID	Airtable Search	Extrae el número de cotización del asunto del mail (regex sobre "COT-n", tolerante al prefijo "Re:" que agrega Gmail) y busca el registro.

Nodo	Tipo	Función
IA - Extraer Precio de Respuesta (Claude)	HTTP Request	Lee el snippet del mail del proveedor y devuelve el precio cotizado en JSON estructurado.
Buscar Proveedor por Mail	Airtable Search	Identifica qué proveedor respondió, extrayendo solo la dirección de mail del remitente (aunque venga con nombre delante).
Preparar Precio y Estado (Procesado por IA)	Set	Parsea el JSON de Claude y arma Precio del repuesto, Proveedor y Estado="Procesado por IA".
Airtable - Estado: Procesado por IA	Airtable Update	Escribe precio, proveedor y estado en el registro.
HITL - Notificar Validación de Precio	Slack (sendAndWait)	Segundo punto de control humano: muestra precio y proveedor concretos, y pausa hasta que se aprueba o rechaza.
¿Se aprobó el precio?	IF	Evalúa $json.data.approved === true.
Pendiente - Revisión Manual (Precio no aprobado)	NoOp	Rama de rechazo; el registro queda en "Procesado por IA" para contactar otro proveedor.
Preparar id y Estado (Aprobado por Humano)	Set	Arma el estado final tras la aprobación del precio.
Airtable - Estado: Aprobado por Humano	Airtable Update	Escribe el estado; de acá en más sigue directo a Rama 2, sin depender de un segundo trigger.
Preparar Datos de Mail Final	Set	Arma el cuerpo del mail final, sumando Precio del repuesto + Mano de obra y calculando el total.
Enviar Mail Final al Cliente	Gmail Send	Envía la cotización final real al cliente.
Preparar Cierre de Cotizacion	Set	Arma el id del registro para el update de cierre.
Airtable - Marcar Mail Enviado	Airtable Update	Marca Mail Enviado a Cliente = true, cerrando el ciclo de esa cotización.

Aclaración: dos HITL distintos, que aprueban cosas distintas

El HITL de Flujo A aprueba el **envío de la solicitud de compulsa a los proveedores** (por eso el estado resultante es "Enviado a proveedores"). El HITL de Flujo B/Rama 2 aprueba el **precio cotizado por el proveedor** (por eso el estado resultante es "Aprobado por Humano"). No son el mismo gate ni aprueban lo mismo, aunque comparten la misma mecánica técnica de Slack sendAndWait.

5. Bitácora de fallas reales encontradas en testing

Ambos workflows se probaron nodo por nodo y de punta a punta con datos y ejecuciones reales (no solo revisión visual del diseño). Esto sacó a la luz una serie de errores silenciosos — el flujo se veía correcto en el canvas, pero fallaba o producía datos vacíos apenas se lo corría de verdad. La siguiente tabla consolida los patrones de bug más importantes encontrados y su causa raíz real.

#	Bug / síntoma	Causa raíz y fix
1	Referencias sin ".fields." al leer un Airtable Trigger/Search	El nodo Airtable Trigger y los nodos de búsqueda devuelven el shape crudo de la API (`{id, createdTime, fields:{...}}`). Una expresión como `$json.Estado` es undefined en silencio; el valor real está en `$json.fields.Estado`. Corregido en más de 10 nodos entre ambos flujos.
2	Campos guardados en modo "Fixed" en vez de "Expression"	El texto lucía como una expresión (`{{ ... }}`) pero el selector Fixed/Expression del campo nunca se había activado, así que n8n mandaba el string literal en vez de evaluarlo — Airtable y Gmail rechazaban el valor. Causa raíz real detrás de varios "invalid request" que parecían errores de la API externa.
3	Campos Número/Array con "=" pero sin envolver en {{ }}	Un matiz distinto del bug anterior: en campos de tipo Número o Formación, el símbolo "=" siempre está presente, pero si el código no está envuelto en `{{ }}`, tampoco se evalúa — se devuelve como texto literal. Encontrado en "Preparar Precio y Estado" (Flujo B) tras dos vueltas de diagnóstico, porque el mensaje de error era idéntico al de un bug distinto ya corregido antes.
4	Nombres de nodo que no coinciden exactamente en referencias cruzadas	Expresiones como `$('Activador de Gmail')` apuntando a un nodo que en realidad se llama "Gmail Trigger", o "Cotización" con tilde contra "Cotizacion" sin tilde en el canvas real. n8n tira "Referenced node doesn't exist" recién al ejecutar ese nodo puntual.
5	Checkbox de Airtable ausente cuando es false	La API de Airtable omite por completo los campos checkbox no marcados — no llegan como false, faltan. Rompía comparaciones "is false"; se corrigió comparando explícitamente `=== true` y tratando la ausencia como falso.
6	Respuesta de Claude envuelta en bloque de código Markdown	El prompt pedía JSON plano pero Claude devolvía ```json ... ``` alrededor, rompiendo el `JSON.parse` posterior. Corregido con una regex que limpia el bloque de código antes de parsear (fix puntual; la sección 6 explica la solución arquitectónica de fondo).
7	Filtro vacío en "Buscar Proveedores" mandaba el mismo pedido a todos	El campo de fórmula había quedado vacío, así que la búsqueda devolvía todos los proveedores sin importar la marca del vehículo — un Renault y un Nissan recibían el mismo pedido sin importar el auto. Corregido con un filtro por fórmula que compara la marca del proveedor contra la del vehículo (o "Multimarca").
8	Mano de obra faltante en el mail final al cliente	No era un bug de mecánica de n8n sino un gap de lógica de negocio: el cuerpo del mail solo mostraba el precio del repuesto. Se extendió la expresión para sumar Mano de obra y mostrar el total.
9	Referencia rota tras eliminar nodos en el rediseño de Rama 2	Al borrar los dos nodos viejos del mecanismo manual (Airtable Trigger de polling + IF redundante), "Preparar Cierre de Cotizacion" quedó referenciando uno de los nodos eliminados. Se corrigió apuntando al nodo más cercano de la nueva cadena. Riesgo clásico de refactor: borrar un nodo sin buscar antes quién lo referencia por nombre en el resto del flujo.

6. Buenas prácticas y lecciones técnicas de n8n

Más allá de los bugs puntuales, el testing de punta a punta dejó un conjunto de prácticas concretas que se aplicaron de forma consistente en ambos flujos y que vale la pena dejar documentadas como checklist para el próximo workflow:

- **Nunca confiar en "la expresión tiene el = así que ya está bien":** verificar cada corrección ejecutando el nodo ("Execute step") con datos reales, no solo revisando el texto guardado.
- **Preferir Structured Output / schema sobre pedir JSON por prompt:** la forma robusta de evitar que Claude envuelva la respuesta en bloques de código es forzar el formato de salida (ver nota de migración más abajo), no solo insistir en el prompt.
- **Después de borrar cualquier nodo, buscar su nombre en el resto del workflow:** n8n no avisa de referencias rotas hasta que se ejecuta ese nodo puntual.
- **Probar un nodo sendAndWait solo dentro de una ejecución real y completa**, nunca aislado con "Execute step": un test de un solo nodo no queda registrado como ejecución pendiente resumible, así que no hay nada que retomar cuando llega la respuesta del botón.
- **"Incluir otros campos de entrada" en un nodo Set, seguido de un Airtable en modo Map Automatically, es una combinación peligrosa:** arrastra campos crudos (createdTime, fields, model, role) que Airtable no reconoce como nombres de campo válidos y rechaza la escritura.
- **Todo campo de Cotizacion que dependa de un link necesita su lookup de texto legible** antes de usarse en un prompt de IA o un mensaje de Slack — nunca leer el link crudo.

Nota de migración sugerida: nodos AI Agent + Structured Output Parser

Los dos nodos HTTP Request crudos a Claude ("IA - Prepara Compulsa" e "IA - Extraer Precio de Respuesta") podrían migrarse a los nodos nativos **AI Agent** de n8n con un **Structured Output Parser** configurado por JSON Schema. Esto reemplaza la limpieza de Markdown por prompt/regex (bug #6 de la sección 5) por una validación de formato garantizada por el propio nodo. Se recomienda deliberadamente *no* darle a estos agentes acceso de herramienta (tool-calling) a los nodos de Airtable/Slack del flujo: el diseño actual, con ramas IF explícitas y dos gates HITL fijos, es una fortaleza de auditabilidad en un proceso de seguros — un agente eligiendo su propio camino de llamadas a herramientas la reduciría.

7. Estado final y checklist de cumplimiento

Requisito	Estado	Evidencia
Flujo A activo y publicado en n8n, con último cambio verificado (activeVersionId == versionId)	**Cumplido**	Versión 08e759b4…
Flujo B activo y publicado en n8n, con último cambio verificado	**Cumplido**	Versión 4bf87208…
Campos de estado (Pendiente / Procesado por IA / Aprobado por Humano, y más granulares)	**Cumplido**	Campo Estado, 7 valores
Guard anti-duplicado para no reprocesar la misma cotización	**Cumplido**	IF post-trigger, Flujo A
Dos puntos de control humano (HITL) antes de acciones irreversibles	**Cumplido**	Slack sendAndWait x2
Filtro de proveedores por marca del vehículo (no un envío masivo indiscriminado)	**Cumplido**	filterByFormula
Mail final al cliente con repuesto + mano de obra + total	**Cumplido**	Preparar Datos de Mail Final
Cierre automático de la cotización sin edición manual de Airtable	**Cumplido**	Rediseño Rama 2 (sendAndWait)
Testing de punta a punta con ejecuciones reales (no solo revisión de diseño)	**Cumplido**	Sección 5, 9+ bugs reales
Interfaz pública de Airtable para no-técnicos	**Pendiente**	Requiere acción manual de Gonzalo
Exports/capturas del canvas real de n8n para el repositorio	**Pendiente**	Sin acceso a n8n en esta sesión

Pendientes que no bloquean la entrega, a resolver por Gonzalo: activar el toggle de publicación de la Interface de Airtable (acción manual, sin API expuesta), y — si se quiere adjuntar como evidencia adicional — exportar desde el propio editor de n8n el JSON real de ambos workflows publicados, dado que esta sesión de trabajo no tiene credenciales ni acceso en vivo a la instancia de n8n del alumno.